

Selenium & Playwright

Complete Test Automation Training

.NET | xUnit/MSTest | CI/CD | Page Object Model

Duration: ~2 Hours | Level: Beginner → Intermediate

1

Agenda

01	Selenium & Playwright Intro	25 min
02	Introduction to Playwright	15 min
03	Manual Testing Fundamentals	10 min
04	Advanced Selenium	20 min
05	Page Object Model & Framework	20 min
06	Automation Frameworks	15 min
07	CI/CD Integration	15 min

1

2

MODULE 1

Selenium & Playwright Introduction

3

What is Test Automation?

- Automated testing uses software to execute test cases without manual effort
- Reduces repetitive regression work – free humans for exploratory testing
- Faster feedback loops → ship with confidence
- Selenium: open-source browser automation framework (since 2004)
- Playwright: modern browser automation by Microsoft (2020+)
- Both support .NET (C#) and cross-browser testing

2

4

Setting Up Selenium in .NET

- Install NuGet: Selenium.WebDriver
- Install NuGet: Selenium.WebDriver.ChromeDriver (match browser version)
- Target framework: .NET 6+ or .NET Framework 4.7+
- Test runner: xUnit or MSTest (both work great with Selenium)
- WebDriverManager NuGet: auto-manages driver binaries
- Recommended IDE: Visual Studio 2022 or VS Code with C# extension

5

Your First ChromeDriver Instance

C#

```
// Install: Selenium.WebDriver, Selenium.WebDriver.ChromeDriver
using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;

IWebDriver driver = new ChromeDriver();
driver.Navigate().GoToUrl("https://example.com");

Console.WriteLine(driver.Title);    // "Example Domain"

driver.Quit();    // Always quit - releases browser process
```

3

6

Locator Strategies

- ID → `driver.FindElement(By.Id("username"))`
- Name → `driver.FindElement(By.Name("q"))`
- Class → `driver.FindElement(By.ClassName("btn-primary"))`
- Tag Name → `driver.FindElement(By.TagName("h1"))`
- CSS Selector → `driver.FindElement(By.CssSelector("#login > .btn"))`
- XPath → `driver.FindElement(By.XPath("//input[@type='submit']"))`
- Link Text → `driver.FindElement(By.LinkText("Sign In"))`

7

Locator Priority Guide

- 1st choice – ID (unique, fast, stable)
- 2nd choice – CSS Selector (readable, widely supported)
- 3rd choice – Name / Data attributes (e.g. data-testid)
- 4th choice – XPath (fallback for complex DOM traversal)
- Avoid brittle locators: absolute XPaths, position-based selectors
- Use browser DevTools → Right-click element → Copy selector
- Validate XPath in browser console: `$x('//input[@id="q"]')`

4

8

Element Interactions

C#

```

IWebElement searchBox = driver.FindElement(By.Name("q"));

// Type text
searchBox.SendKeys("Selenium C#");

// Clear input
searchBox.Clear();

// Click button
driver.FindElement(By.Id("btnSearch")).Click();

// Get text / attribute
string text = driver.FindElement(By.TagName("h1")).Text;
string href = driver.FindElement(By.TagName("a")).GetAttribute("href");

// Check state
bool isEnabled = searchBox.Enabled;
bool isSelected = driver.FindElement(By.Id("agree")).Selected;

```

9

First Complete Selenium Test (xUnit)

C#

```

using OpenQA.Selenium;
using OpenQA.Selenium.Chrome;
using Xunit;

public class GoogleSearchTests : IDisposable {
    private readonly IWebDriver _driver;

    public GoogleSearchTests() {
        _driver = new ChromeDriver();
        _driver.Manage().Window.Maximize();
    }

    [Fact]
    public void SearchGoogle_ReturnsResults() {
        _driver.Navigate().GoToUrl("https://www.google.com");
        _driver.FindElement(By.Name("q")).SendKeys("Selenium C# tutorial");
        _driver.FindElement(By.Name("q")).Submit();
        Assert.Contains("Selenium", _driver.Title);
    }

    public void Dispose() => _driver.Quit();
}

```

5

10

MODULE 2

Introduction to Playwright

11

Selenium vs Playwright

Selenium

- Mature ecosystem (2004)
- WebDriver protocol (W3C)
- Requires separate driver binaries
- Implicit/Explicit waits needed
- Multi-language support
- Larger community & plugins
- Selenium Grid for parallel runs

Playwright

- Modern API (2020) by Microsoft
- CDP + WebSocket protocol
- Browsers bundled (no drivers!)
- Auto-waiting built in
- Faster test execution
- Built-in tracing & video capture
- Parallel by default

6

12

Setting Up Playwright for .NET

- Install NuGet: Microsoft.Playwright
- Run CLI to install browsers: playwright install
- Supported browsers: Chromium, Firefox, WebKit (Safari)
- Works with NUnit, xUnit, MSTest
- Install NuGet: Microsoft.Playwright.NUnit (recommended)
- Playwright generates code via codegen tool: playwright codegen
- No separate driver binary management needed

13

Basic Playwright Operations

C#

```
using Microsoft.Playwright;
using Microsoft.Playwright.NUnit;

[Parallelizable(ParallelScope.Self)]
[TestFixture]
public class PlaywrightDemoTests : PageTest {

    [Test]
    public async Task NavigateAndSearch() {
        await Page.GotoAsync("https://playwright.dev");
        await Page.Locator("text=Get Started").ClickAsync();
        await Expect(Page).ToHaveTitleAsync("Installation | Playwright");
    }

    [Test]
    public async Task FillForm() {
        await Page.GotoAsync("https://example.com/login");
        await Page.FillAsync("#username", "testuser");
        await Page.FillAsync("#password", "secret");
        await Page.ClickAsync("button[type=submit]");
        await Expect(Page.Locator(".welcome")).ToBeVisibleAsync();
    }
}
```

7

14

MODULE 3

Manual Testing Fundamentals

15

Manual Testing Core Concepts

- Manual testing: humans execute test cases step-by-step
- Exploratory testing: simultaneous learning, design, and execution
- Black-box testing: test without knowledge of internal code
- White-box testing: test with code/structure knowledge
- Regression testing: ensure existing features still work after changes
- Smoke testing: quick sanity check – does the app start and basic flow work?
- Sanity testing: focused subset after a bug fix or small change

8

16

Test Case Documentation

- Test Case ID – unique identifier (e.g. TC_LOGIN_001)
- Test Title – clear, action-oriented description
- Preconditions – setup required before execution
- Test Steps – numbered, specific, reproducible actions
- Expected Result – what should happen at each step
- Actual Result – what actually happened (filled during execution)
- Pass/Fail/Blocked – final status with date and tester name

17

Sample Test Case Template

C#

```
Test Case ID : TC_LOGIN_001
Title       : Verify successful login with valid credentials
Module      : Authentication
Priority     : High
Preconditions: User account exists (user@test.com / Pass@123)
```

Steps:

1. Navigate to <https://app.example.com/login>
2. Enter email: user@test.com
3. Enter password: Pass@123
4. Click "Login" button

Expected Result:

- User is redirected to /dashboard
- Welcome message displayed: "Hello, Test User"
- Session cookie set

Actual Result: [Fill during execution]

Status: [] Pass [] Fail [] Blocked

9

18

Test Flow Design

- Happy path: test the most common expected user journey
- Negative path: invalid input, missing fields, wrong credentials
- Edge cases: boundary values, empty states, maximum lengths
- Decision table: map input combinations to expected outputs
- State transition: test state changes (e.g. order: Draft→Pending→Complete)
- Use-case driven: design tests from user stories / acceptance criteria
- Document as flowcharts for complex multi-step flows

19

MODULE 4

Advanced Selenium

10

20

Handling Dropdowns

C#

```
using OpenQA.Selenium.Support.UI;

IWebElement ddl = driver.FindElement(By.Id("country"));
SelectElement select = new SelectElement(ddl);

// Three ways to select
select.SelectByText("India");
select.SelectByValue("IN");
select.SelectByIndex(2);

// Read options
string selected = select.SelectedOption.Text;
IList<IWebElement> allOptions = select.Options;

// Multi-select
select.SelectByText("India");
select.SelectByText("USA"); // Works if multiple="multiple"
```

21

Alerts, Windows & Frames

C#

```
// — Alerts —————
IAlert alert = driver.SwitchTo().Alert();
string msg = alert.Text;
alert.Accept(); // OK
alert.Dismiss(); // Cancel
alert.SendKeys("input for prompt");

// — New Windows / Tabs —————
string mainWindow = driver.CurrentWindowHandle;
driver.SwitchTo().Window(driver.WindowHandles.Last());
driver.Close();
driver.SwitchTo().Window(mainWindow);

// — Frames —————
driver.SwitchTo().Frame("frameName");
driver.SwitchTo().Frame(driver.FindElement(By.Id("myFrame")));
driver.SwitchTo().DefaultContent(); // back to main page
```

11

22

Waits – Implicit, Explicit, Fluent

C#

```
// Implicit Wait (global - applies to all FindElement calls)
driver.Manage().Timeouts().ImplicitWait = TimeSpan.FromSeconds(10);

// Explicit Wait (targeted condition)
WebDriverWait wait = new WebDriverWait(driver, TimeSpan.FromSeconds(15));
IWebElement el = wait.Until(ExpectedConditions.ElementIsVisible(By.Id("result")));

// Fluent Wait (custom polling)
DefaultWait<IWebDriver> fluentWait = new DefaultWait<IWebDriver>(driver) {
    Timeout = TimeSpan.FromSeconds(20),
    PollingInterval = TimeSpan.FromMilliseconds(500)
};
fluentWait.IgnoreExceptionTypes(typeof(NoSuchElementException));
IWebElement result = fluentWait.Until(d => d.FindElement(By.Id("data")));
```

23

Actions Class & JavaScript Executor

C#

```
using OpenQA.Selenium.Interactions;

Actions actions = new Actions(driver);

// Mouse hover
actions.MoveToElement(driver.FindElement(By.Id("menu"))).Perform();

// Drag and drop
IWebElement src = driver.FindElement(By.Id("draggable"));
IWebElement tgt = driver.FindElement(By.Id("droppable"));
actions.DragAndDrop(src, tgt).Perform();

// Right-click
actions.ContextClick(driver.FindElement(By.Id("item"))).Perform();

// JavaScript Executor
IJavaScriptExecutor js = (IJavaScriptExecutor)driver;
js.ExecuteScript("arguments[0].click();", element); // force click
js.ExecuteScript("arguments[0].scrollIntoView(true);", el); // scroll to
string title = (string)js.ExecuteScript("return document.title;");
```

12

24

Taking Screenshots

C#

```
using OpenQA.Selenium;

// Full page screenshot
Screenshot shot = ((ITakesScreenshot)driver).GetScreenshot();
shot.SaveAsFile("screenshot.png");

// Element screenshot (Selenium 4+)
IWebElement banner = driver.FindElement(By.Id("hero-banner"));
Screenshot elShot = ((ITakesScreenshot)banner).GetScreenshot();
elShot.SaveAsFile("banner.png");

// In xUnit - save on test failure
public void Dispose() {
    if (TestContext.CurrentTestOutcome == UnitTestOutcome.Failed) {
        var sc = ((ITakesScreenshot)_driver).GetScreenshot();
        sc.SaveAsFile($"fail_{DateTime.Now:yyyyMMdd_HH:mm:ss}.png");
    }
    _driver.Quit();
}
```

25

Playwright Auto-Waiting

- Playwright waits automatically for elements to be: visible, stable, enabled
- No Thread.Sleep() or explicit WebDriverWait needed for most cases
- await Page.ClickAsync() – waits until element is actionable
- await Page.FillAsync() – waits for editable + focused state
- await Expect(locator).ToBeVisibleAsync() – assertion with retry
- Configure timeout: Page.SetDefaultTimeout(30_000) (milliseconds)
- Network idle: await Page.WaitForLoadStateAsync(LoadState.NetworkIdle)

13

26

MODULE 5

Page Object Model & Framework Design

27

Design Patterns in Test Automation

- Page Object Model (POM) – encapsulate page elements + actions in a class
- Fluent Interface – chain methods for readable test syntax
- Factory Pattern – create WebDriver instances centrally
- Singleton – one driver instance per test class lifecycle
- Builder Pattern – construct complex test data objects
- Strategy Pattern – swap locator strategies at runtime
- Benefits: maintainability, reusability, readability, DRY principle

1 4

28

Base Page Class

C#

```

public abstract class BasePage {
    protected IWebDriver Driver;
    protected WebDriverWait Wait;

    protected BasePage(IWebDriver driver) {
        Driver = driver;
        Wait = new WebDriverWait(driver, TimeSpan.FromSeconds(15));
    }

    protected IWebElement FindElement(By locator) =>
        Wait.Until(ExpectedConditions.ElementIsVisible(locator));

    protected void Click(By locator) => FindElement(locator).Click();
    protected void Type(By locator, string text) {
        var el = FindElement(locator);
        el.Clear();
        el.SendKeys(text);
    }
    protected string GetText(By locator) => FindElement(locator).Text;
}

```

29

Page Object Model – LoginPage

C#

```

public class LoginPage : BasePage {
    // Locators as read-only fields
    private readonly By _usernameField = By.Id("username");
    private readonly By _passwordField = By.Id("password");
    private readonly By _loginButton = By.CssSelector("button[type='submit']");
    private readonly By _errorMessage = By.ClassName("error-msg");

    public LoginPage(IWebDriver driver) : base(driver) { }

    public DashboardPage Login(string username, string password) {
        Type(_usernameField, username);
        Type(_passwordField, password);
        Click(_loginButton);
        return new DashboardPage(Driver);
    }

    public string GetErrorMessage() => GetText(_errorMessage);
}

```

15

30

POM Test with xUnit

C#

```

public class LoginTests : IDisposable {
    private readonly IWebDriver _driver;
    private readonly LoginPage _loginPage;

    public LoginTests() {
        _driver = new ChromeDriver();
        _driver.Navigate().GoToUrl("https://app.example.com/login");
        _loginPage = new LoginPage(_driver);
    }

    [Fact]
    public void ValidLogin_NavigatesToDashboard() {
        var dashboard = _loginPage.Login("user@test.com", "Pass@123");
        Assert.Equal("Dashboard", _driver.Title);
    }

    [Fact]
    public void InvalidLogin_ShowsError() {
        _loginPage.Login("wrong@test.com", "bad");
        Assert.Equal("Invalid credentials", _loginPage.GetErrorMessage());
    }
}

```

31

Test Fixtures & Setup/Teardown

- xUnit: Constructor = Setup, IDisposable.Dispose() = Teardown
- xUnit: IClassFixture<T> – share driver across tests in one class
- xUnit: ICollectionFixture<T> – share across multiple test classes
- MSTest: [TestInitialize] / [TestCleanup] / [ClassInitialize]
- NUnit: [SetUp] / [TearDown] / [OneTimeSetUp] / [OneTimeTearDown]
- Best practice: each test gets a fresh browser state
- Use [Collection] in xUnit to control parallelism

16

32

MODULE 6

Automation Frameworks

33

Framework Comparison

Data-Driven Framework

- Tests read data from external source
- Excel / CSV / JSON / database
- Same test logic, multiple data sets
- Ideal for: form validation, login combos
- [Theory] + [MemberData] in xUnit
- Reduces test count while increasing coverage

Keyword-Driven Framework

- Tests driven by keyword tables
- Keywords: Click, Type, Navigate, Verify
- Non-programmers can write tests
- Keyword engine interprets actions
- More complex to build initially
- High reusability across projects

17

34

Data-Driven with xUnit [Theory]

C#

```
// CSV data file: testdata/login.csv
// user@test.com,Pass@123,true
// bad@user.com,wrong,false

public class LoginDataTests {
    public static IEnumerable<object[]> LoginData =>
        File.ReadAllLines("testdata/login.csv")
            .Select(line => line.Split(','))
            .Select(p => new object[] { p[0], p[1], bool.Parse(p[2]) });

    [Theory]
    [MemberData(nameof(LoginData))]
    public void Login_WithVariousCredentials(
        string email, string password, bool expectSuccess) {
        using var driver = new ChromeDriver();
        var page = new LoginPage(driver);
        page.Navigate();
        var result = page.Login(email, password);
        Assert.Equal(expectSuccess, result.IsSuccess);
    }
}
```

35

Hybrid Framework

- Combines Data-Driven + Keyword-Driven + POM
- POM for page abstraction and element management
- Data-driven for test data (Excel / JSON configs)
- Keyword layer for business actions (Login, PlaceOrder, Verify)
- Reporting layer: ExtentReports or Allure
- Config layer: appsettings.json for environment URLs, timeouts
- Most enterprise frameworks are hybrid by nature

18

36

MODULE 7

CI/CD Integration

37

Azure DevOps Pipeline (azure-pipelines.yml)

C#

```
trigger:
- main

pool:
  vmImage: 'ubuntu-latest'

steps:
- task: UseDotNet@2
  inputs:
    version: '8.x'

- script: dotnet restore
  displayName: 'Restore NuGet packages'

- script: dotnet build --no-restore
  displayName: 'Build solution'

- script: dotnet test --no-build --logger trx
  displayName: 'Run Selenium tests'

- task: PublishTestResults@2
  inputs:
    testResultsFormat: 'JUnit'
```

19

38

ExtentReports Integration (C#)

C#

```
using AventStack.ExtentReports;
using AventStack.ExtentReports.Reporter;

// In test class or fixture:
ExtentReports extent = new ExtentReports();
extent.AttachReporter(new ExtentSparkReporter("TestReport.html"));

ExtentTest test = extent.CreateTest("Login Test");

try {
    // ... test steps ...
    test.Pass("Login successful - dashboard loaded");
} catch (Exception ex) {
    var shot = ((ITakesScreenshot)driver).GetScreenshot();
    shot.SaveAsFile("fail.png");
    test.Fail(ex.Message)
        .AddScreenCaptureFromPath("fail.png");
} finally {
    extent.Flush(); // Write HTML report
}
```

39

Selenium Grid – Overview

- Selenium Grid enables distributed parallel test execution
- Hub: central node that receives test requests and routes them
- Node: machine with browser(s) where tests actually run
- Selenium Grid 4: uses GraphQL API, Docker-friendly
- Start hub: `java -jar selenium-server.jar hub`
- Start node: `java -jar selenium-server.jar node`
- Set RemoteWebDriver with hub URL in your test setup
- Cloud alternatives: BrowserStack, Sauce Labs, LambdaTest

20

40

RemoteWebDriver for Grid

C#

```
// Connect to Selenium Grid Hub
Uri gridHub = new Uri("http://localhost:4444");

ChromeOptions options = new ChromeOptions();
options.AddArgument("--headless");

IWebDriver driver = new RemoteWebDriver(gridHub, options.ToCapabilities());
driver.Navigate().GoToUrl("https://example.com");

// For cloud grids (BrowserStack example)
var bsOptions = new Dictionary<string, object> {
    { "os", "Windows" }, { "os_version", "11" },
    { "browser", "Chrome" }, { "browser_version", "latest" }
};
ChromeOptions cloudOptions = new ChromeOptions();
cloudOptions.AddAdditionalOption("bstack:options", bsOptions);
IWebDriver cloud = new RemoteWebDriver(
    new Uri("https://hub-cloud.browserstack.com/wd/hub"), cloudOptions);
```

41

Key Takeaways

- Selenium: mature, flexible, WebDriver standard – great for legacy projects
- Playwright: modern, fast, built-in auto-wait – ideal for greenfield
- Page Object Model keeps tests maintainable as apps grow
- Data-Driven + POM + Reporting = production-grade hybrid framework
- CI/CD integration closes the feedback loop – automate everything

21

Questions? Practice makes perfect – run the examples!

42